

PROJECT REPORT: TASK-2

Embedded Systems Lab Setup & Sensor Interfacing

Intern Name: Promoredhan K G

Institution: Sri Ramakrishna Institute of Technology, Coimbatore

Company: Maincrafts Technology

Task: Task-2 - Embedded Systems & IoT

1. Executive Summary

This report documents the successful completion of Task-2 for the Embedded Systems & IoT internship. The primary objective was to establish a working embedded development environment, interface a standard sensor with a microcontroller, and write the necessary C/C++ firmware to process real-time data and trigger an automated response. This project utilizes a Passive Infrared (PIR) motion sensor to detect movement and activate an LED visual alert.

2. Working Environment & Methodology

To complete this task without physical hardware, a virtual simulation approach was selected.

- **Simulator Environment:** Tinkercad Circuits
- **Microcontroller:** Arduino UNO R3 (ATmega328P based)
- **Programming Language:** Embedded C/C++

Methodology: The circuit was first designed and routed on a virtual breadboard. Once the hardware pathways were verified, the logic was programmed in the integrated text editor, compiled, and executed within the simulation environment to verify real-time responsiveness.

3. Hardware Components & Circuit Architecture

The circuit relies on three main components wired to the Arduino UNO. Proper grounding and power distribution were maintained to simulate industry-standard breadboard prototyping.

Component	Terminal	Arduino Connection	Technical Purpose
PIR Motion Sensor	Power (VCC)	5V Pin	Supplies the necessary operating voltage to the sensor's internal logic.
	Ground (GND)	GND Pin	Completes the circuit for the sensor.
	Signal (OUT)	Digital Pin 2	Transmits a digital HIGH (5V) or LOW (0V) signal to the microcontroller based on motion detection.
LED (Red)	Anode (+)	Digital Pin 13	Acts as the visual output. Connected in series with a resistor.
	Cathode (-)	GND Pin	Completes the LED circuit path.
Resistor (220 Ω)	Terminals 1 & 2	Between Pin 13 & LED	Limits the electrical current flowing through the LED to prevent component burnout.

4. Software Implementation & Firmware Logic

The firmware is designed to continuously poll the PIR sensor for data. It utilizes serial communication for debugging purposes and digital I/O for hardware control.

```

// --- Pin Definitions ---
int pirSensorPin = 2;    // The Digital Input pin reading the PIR sensor
int ledPin = 13;        // The Digital Output pin controlling the LED
int motionStatus = 0;    // Variable to hold the current state of the
sensor

void setup() {
    // --- Initialization ---
    pinMode(ledPin, OUTPUT);    // Configures Pin 13 to send power OUT
    pinMode(pirSensorPin, INPUT); // Configures Pin 2 to read power IN

    // Initialize Serial Monitor at 9600 bits per second for logging
    Serial.begin(9600);
}

void loop() {
    // --- Main Execution Loop ---
    // 1. Read the current digital state of the PIR sensor (HIGH or LOW)
    motionStatus = digitalRead(pirSensorPin);

    // 2. Control Logic Threshold
    if (motionStatus == HIGH) {
        // If motion is detected:
        digitalWrite(ledPin, HIGH);    // Supply 5V to the LED to
turn it ON
        Serial.println("ALERT: Motion detected!"); // Log the event to the
system monitor
        delay(1000);    // Pause for 1 second to
debounce the alert
    } else {
        // If no motion is detected:
        digitalWrite(ledPin, LOW);    // Cut power to the LED to
turn it OFF
    }
}

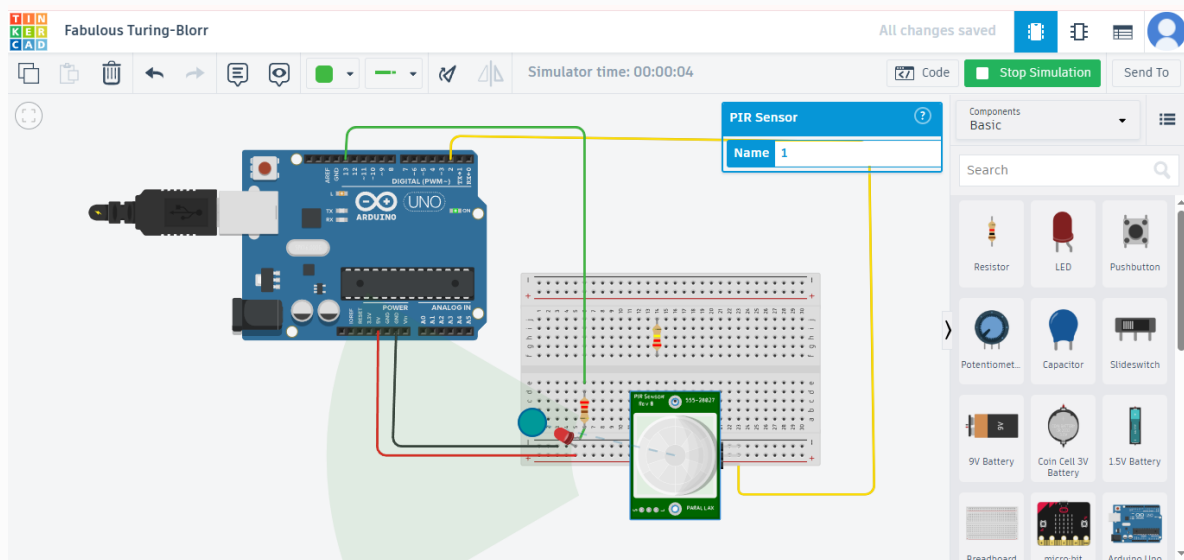
```

5. Detailed Technical Explanation

As required by the task deliverables, the following explains the fundamental operations of the hardware and software layers:

- **Hardware Operation (How the Sensor Works):** The PIR (Passive Infrared) sensor consists of a pyroelectric sensor that detects levels of infrared radiation. Every living being emits heat in the form of infrared radiation. When a human walks into the sensor's field of view, the sensor detects a sudden spike in infrared energy. The sensor's onboard circuitry processes this differential change and outputs a digital HIGH signal (typically 3.3V or 5V) through its OUT pin to alert the connected microcontroller.
- **Software Operation (What the Code Does):** The Arduino firmware acts as the central brain. During the `setup()` phase, it registers the PIR sensor as an INPUT device and the LED as an OUTPUT device. During the continuous `loop()` phase, the microcontroller acts as a monitor, constantly checking the voltage on Digital Pin 2. When the PIR sensor sends a HIGH voltage (indicating motion), the if statement evaluates to true. The Arduino then executes a `digitalWrite()` command to send 5V out of Pin 13, illuminating the LED, and prints an alert message. When the person leaves the area, the sensor drops back to LOW, and the Arduino shuts off the LED.

6. Circuit Diagram



7. Conclusion & Next Steps

Through the successful completion of Task-2, the fundamental principles of embedded systems development have been practically applied. The virtual environment is fully operational, thus finished